

Exercício 1: Sistema de Cache Genérico

Contexto Detalhado

Você precisa desenvolver um sistema de cache em memória que seja flexível o suficiente para armazenar diferentes tipos de dados (usuários, produtos, configurações, etc.) mantendo controle sobre expiração e type safety. O cache deve funcionar como uma camada temporária de armazenamento para melhorar performance de aplicações.

Especificações da Interface

1. Interface Cacheable

- Deve garantir que todo item armazenado tenha uma chave única de identificação
- Deve incluir informação temporal para controle de expiração
- Pense em que propriedades são essenciais para qualquer item que possa ser cacheado

Especificações da Classe Genérica

2. Classe SimpleCache<T>;

Armazenamento

- Use uma estrutura interna adequada para mapear chaves aos valores
- Garanta que apenas itens que implementem Cacheable possam ser armazenados
- Considere como combinar o tipo genérico T com a interface Cacheable

Método set()

- Deve aceitar um item que seja tanto do tipo T quanto implemente Cacheable
- Precisa armazenar o item usando sua chave como identificador
- Considere se deve atualizar timestamp automaticamente ou usar o fornecido

Método get()

- Deve buscar um item pela sua chave
- Retorne o tipo correto ou null se não encontrado
- Considere se deve verificar expiração automaticamente neste método

Método isExpired()

- Recebe uma chave e um tempo máximo de vida (em milissegundos)
- Deve verificar se o item existe e se ultrapassou o tempo limite
- Compare o timestamp atual com o timestamp do item + maxAge

Método clear()

- Remove todos os itens do cache
- Deve deixar o cache em estado limpo para novo uso

Tipos de Teste Requeridos

3. Implementações para Teste

Tipo UserData

- Deve representar dados de usuário (id, nome, email)
- Precisa implementar Cacheable adequadamente
- Considere que tipo de timestamp faz sentido para dados de usuário

Tipo ProductInfo

- Deve representar informações de produto (id, título, preço)
- Também deve implementar Cacheable
- Pense em cenários onde produtos precisariam ser cacheados

Funcionalidades Esperadas

Cenários de Uso

- Armazenar dados de usuário após login para evitar consultas frequentes ao banco
- Cachear informações de produtos para catálogos online
- Gerenciar expiração automática para dados sensíveis
- Limpar cache quando necessário (logout, atualizações, etc.)

Considerações de Design

Type Safety

- O sistema deve impedir armazenamento de tipos incompatíveis
- Métodos genéricos devem manter a tipagem correta
- IntelliSense deve funcionar corretamente com os tipos definidos

Flexibilidade

- O mesmo cache deve funcionar com qualquer tipo que implemente Cacheable
- Deve ser possível ter múltiplas instâncias de cache para diferentes tipos
- Considere como diferentes tipos podem ter diferentes necessidades de expiração

Robustez

- Trate casos onde chaves não existem
- Considere comportamento com timestamps inválidos
- Pense em edge cases como maxAge negativo ou zero

Objetivo: Este exercício foca em conceitos fundamentais de generics com constraints, implementação de interfaces, e design de APIs type-safe em TypeScript.